

EventStoryLine 项目结构费曼版

用厨房、档案馆和考试卷把这个仓库讲明白

yifeng_study

2026 年 5 月 7 日



图 1: 费曼式第一句话：这个项目像一间故事线厨房，语料是食材，脚本是厨师，评测分数是试吃结果。

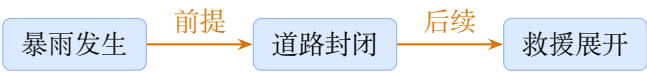
目录

1	先用一句大白话讲清楚	2
2	把仓库想成档案馆	2
3	XML 标注到底是什么	3
4	三种数据版本像三本教材	4
5	标准答案是怎么做出来的	4
6	基线脚本像学生做题	5

目录	2
7 评测脚本像老师批卷	6
8 建议你按这个顺序读	6
9 一张总复习图	7
10 打开方式	7

1 先用一句大白话讲清楚

这个仓库在做一件事：**把新闻里的事件找出来，再判断这些事件之间有没有“故事线关系”。**
比如一篇新闻里出现了这些事件：



项目里的数据和脚本，就是在回答这些问题：

- 哪些词是事件？
- 两个看起来不同的事件提及，是不是其实指同一件事？
- 哪些事件之间有“前因后果”或“情节推进”关系？
- 一个系统猜出的答案，和标准答案比，准不准？

2 把仓库想成档案馆



图 2: 目录不是随机堆放的：它更像一个档案馆，每个房间放一种材料。

真实目录或文件	费曼比喻	你该怎么理解
ECB+_LREC2014/	原始馆藏	原始 ECB+ 语料，像还没加工的新闻档案。
annotated_data/	贴好便签的档案	ESC 标注数据，里面标了事件和情节关系。

evaluation_format/	考试标准答案柜台	已经整理成评测脚本容易读取的格式。
ecb+_naf/	语言学工具卡片	提供 lemma、句子等信息，PPMI 基线会用。
ppmi_events_ecb+_test/	实验材料盒	PPMI 基线相关的测试或中间材料。
create_gold_document.py	档案整理员	把原始标注整理成标准答案文件。
baseline_*.py	答题学生	用规则猜事件关系，生成系统答案。
eval_script.py	批卷老师	对照标准答案，计算精确率、召回率、F1。

3 XML 标注到底是什么

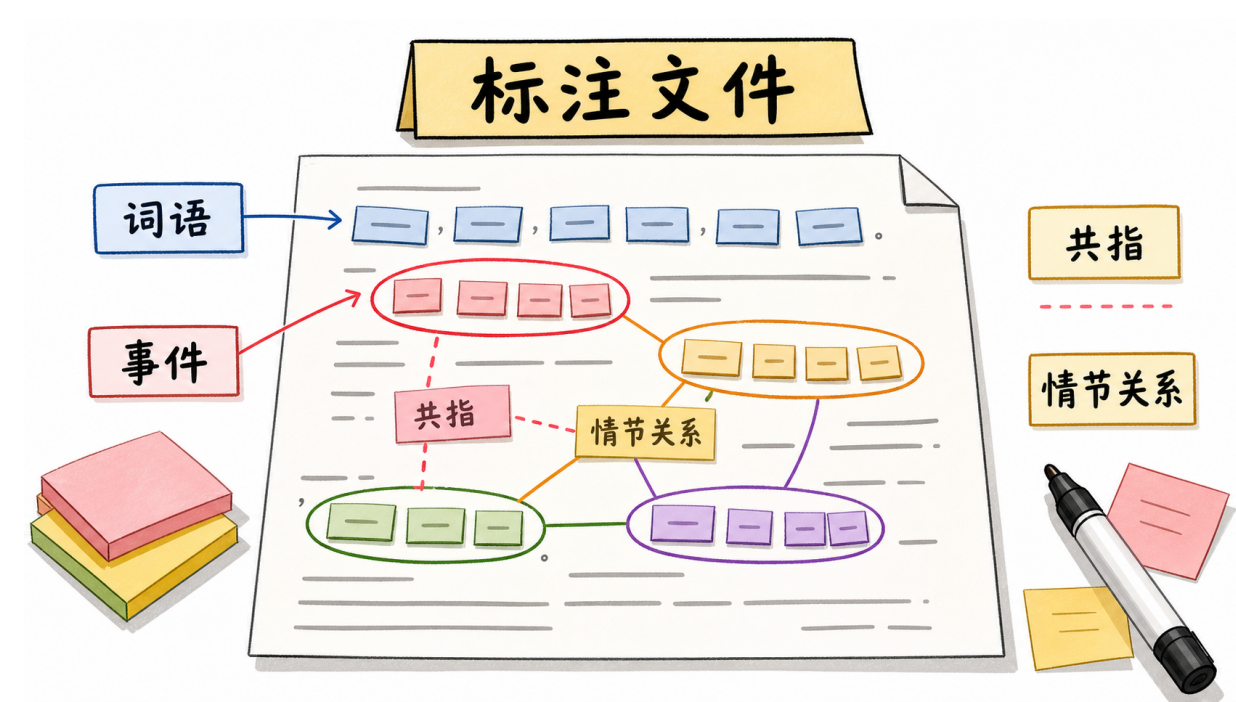


图 3: XML 标注可以理解成“贴满便签的新闻稿”：哪些词是事件，哪些事件互相有关，都写在便签里。

不用先害怕 XML。你可以把一个 XML 文件想成一张新闻稿，旁边贴了很多便签：

- **token**：新闻稿里的每个词，像一个小卡片。
- **Markables**：哪些词被圈出来了，比如“爆炸”“逮捕”“宣布”。
- **Relations**：被圈出来的东西之间有什么关系。
- **PLOT_LINK**：故事线关系，比如一个事件是另一个事件的前提或后续。

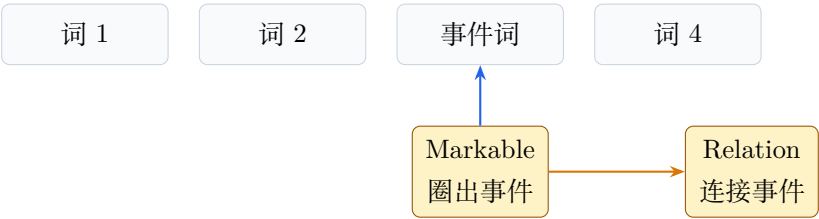


图 4: XML 的关键不是尖括号，而是“词、事件标注、关系标注”三层。

4 三种数据版本像三本教材

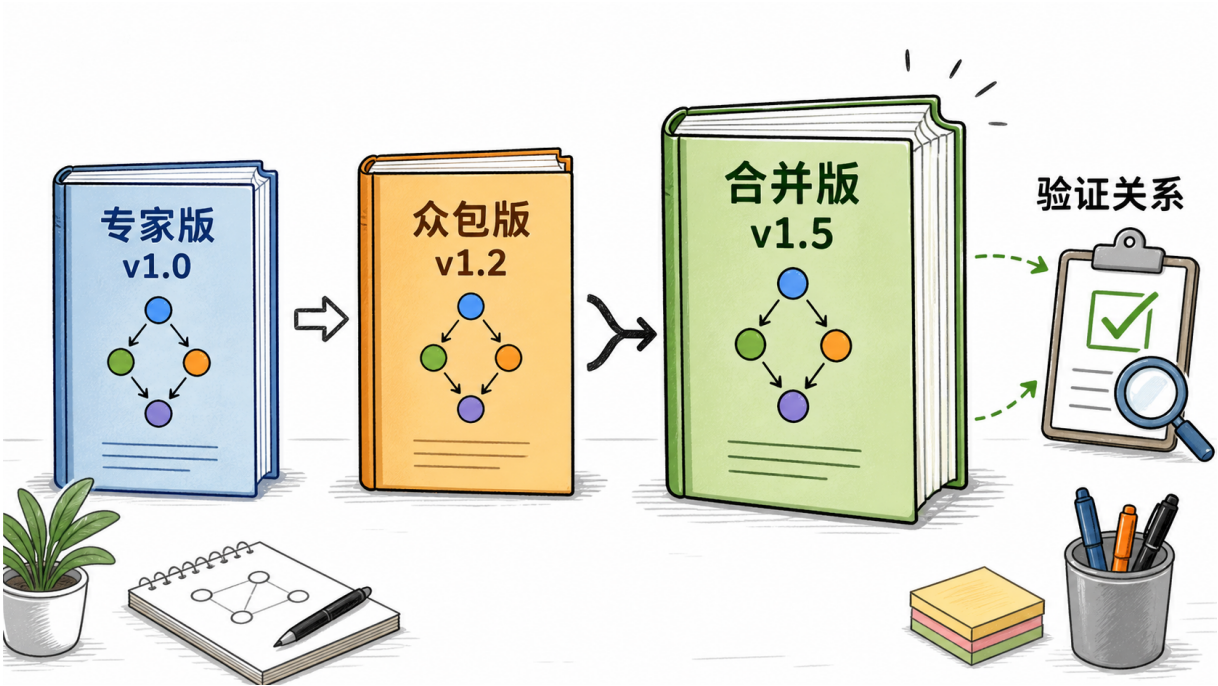


图 5: v1.0、v1.2、v1.5 可以先理解成三本不同来源的故事线教材。

- v1.0: 专家版。可以理解成老师亲自批注的版本。
- v1.2: 众包版。可以理解成很多同学一起标注后汇总的版本。
- v1.5: 合并版。把专家和众包信息结合起来，README 里提到还带有 `origin` 和 `validated` 这样的属性。

如果你只是学习项目结构，建议先看 v1.5，因为它更完整。

5 标准答案是怎么做出来的

`create_gold_document.py` 像一个档案整理员。它同时打开两类档案：

- ECB+ 原始文件：告诉你事件 `mention` 和共指链。
- ESC 标注文件：告诉你事件之间的 `PLOT_LINK`。

然后它做一件很重要的事：**把一个事件关系扩展到共指链里的更多事件提及对。**

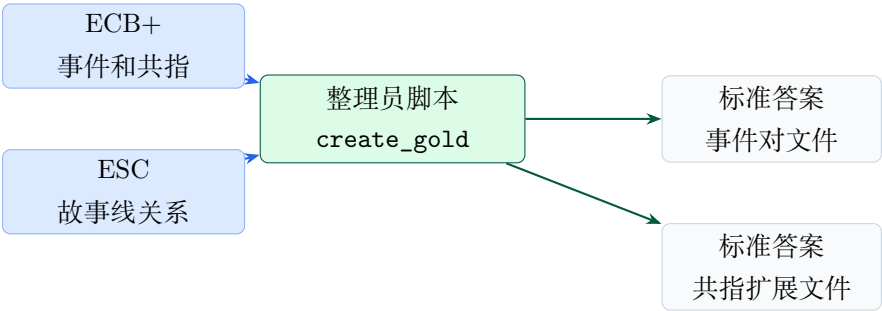


图 6: 标准答案不是凭空来的，而是由 ECB+ 的共指信息和 ESC 的故事线标注合并出来的。

6 基线脚本像学生做题

基线系统不是神经网络模型。它更像学生用一套简单规则做题：

- `baseline_OP.py`: 看到同一句里有多个事件，就把它们两两连起来。
- `baseline_PPMI1.py`: 看事件词的共现强度，觉得常一起出现的事件更可能有关系。
- `baseline_PPMI_CONTAINS.py`: 在 PPMI 的基础上，又看这些事件是不是被同一个时间锚装在一起。



图 7: 三类基线可以按“简单规则、统计规则、时间增强规则”来理解。

7 评测脚本像老师批卷



图 8: 系统输出像学生答案，标准答案像参考答案，评测脚本就是批卷老师。

`eval_script.py` 做的事情很朴素：

1. 读标准答案文件。
2. 读系统输出文件。
3. 看系统猜对了多少、猜错了多少、漏掉了多少。
4. 算精确率、召回率、F1。

精确率
猜出来的有多少是真的

召回率
真的里面找回了多少

F1
两者的折中分

8 建议你按这个顺序读

1. 先看评测文件：打开 `evaluation_format/test_corpus/v1.5/event_mentions_extended/`，感受一行三列长什么样。
2. 再看批卷老师：读 `eval_script.py`，因为它短，而且会告诉你“系统答案应该长什么样”。
3. 再看最简单学生：读 `baseline_0P.py`，理解同句事件配对。
4. 再看档案整理员：读 `create_gold_document.py`，理解标准答案怎样生成。
5. 最后看高级学生：读 `baseline_PPMI1.py` 和 `baseline_PPMI_CONTAINS.py`。

9 一张总复习图

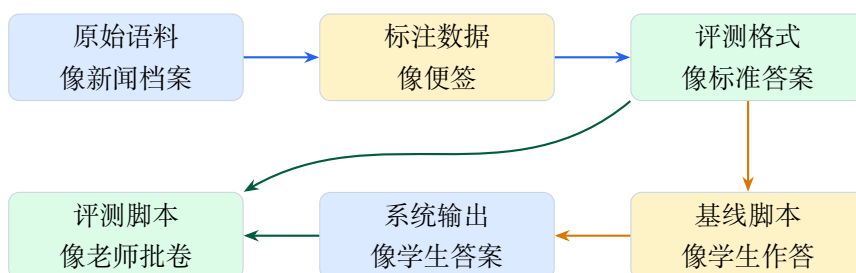


图 9: 记住这条链: 原始语料到标注, 标注到标准答案, 基线生成答案, 评测脚本批改。

10 打开方式

在 VS Code 中打开整个项目:

```
code /Users/luyifeng/Documents/GitHub/EventStoryLine
```

这份费曼版文档在:

```
yifeng_study/feynman_project_guide.tex  
yifeng_study/assets_v2/
```

中文 LaTeX 建议用 XeLaTeX 编译:

```
cd /Users/luyifeng/Documents/GitHub/EventStoryLine/yifeng_study  
xelatex feynman_project_guide.tex
```